

Baseline Reproduction Report

Reduced-scale reproduction of Yao et al. (2023), *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023. Model under test: `z-ai/glm-5.2` (via OpenRouter), in place of the paper's PaLM-540B. Results below are the n=10 gate runs (`results/raw/*.jsonl`); see `analysis/figures/` and `results/summary/accuracy_summary.csv` for the full data.

1. Reproduction vs. Original

HotpotQA (Exact Match, %)

Method	Paper (PaLM-540B)	Reproduction (GLM-5.2, n=10)	Δ
Standard	28.7	30.0	+1.3
CoT	29.4	60.0	+30.6
Act	25.7	70.0	+44.3
ReAct	27.4	60.0	+32.6
CoT-SC	33.4	70.0	+36.6
ReAct→CoT-SC	35.1	60.0	+24.9
CoT-SC→ReAct	34.2	60.0	+25.8

FEVER (Label Accuracy, %)

Method	Paper (PaLM-540B)	Reproduction (GLM-5.2, n=10)	Δ
Standard	57.1	20.0	-37.1
CoT	56.3	40.0	-16.3
Act	58.9	40.0	-18.9
ReAct	60.9	30.0	-30.9
CoT-SC	60.4	50.0	-10.4
ReAct→CoT-SC	62.0	30.0	-32.0
CoT-SC→ReAct	64.6	40.0	-24.0

Observations

- **Absolute deltas are not meaningful at n=10.** A single flipped example moves accuracy by 10pp. These numbers indicate direction and rough scale only — do not read them as a calibrated replication until the n=100 run lands.
- **HotpotQA reasoning/acting methods outperform the paper**, while **FEVER methods uniformly underperform it**. This asymmetry, consistent across all 7 methods within each domain, is more likely a domain-level effect than noise: GLM-5.2 appears substantially stronger than PaLM-540B at multi-hop QA reasoning/retrieval, but weaker (or more label-format-sensitive) at 3-way claim verification. FEVER's SUPPORTS/REFUTES/NOT ENOUGH INFO output space is more constrained than free-text QA, so label-parsing strictness in `metrics.py::fever_acc()` is worth auditing before trusting the FEVER gap.
- **Standard is the closest match to the paper in both domains** (only method within ~1-3pp on HotpotQA, and the smallest relative gap direction-wise on FEVER pattern). This is expected — Standard has no retrieval/tool-use component, so it isolates raw model knowledge from environment-interaction quality, and is the method least sensitive to our WikiEnv reimplementation's fidelity to the original.
- **Both hybrid back-off strategies fire on real data in both domains** (10-30% trigger rates), confirming the back-off logic is live rather than dead code, per `analysis/figures/backoff_rates.png`.
- **Known confound:** one manually-inspected ReAct HotpotQA trajectory (Irene Jacob / Stuart Bird question, logged in `AI_LOG.md`, Milestone 1 gate) showed the model missing its own Observation text and chasing a name variant for all 7 steps — a genuine multi-hop reasoning failure, not a pipeline bug, and a candidate qualitative failure mode worth one paragraph in the final write-up.

Caveat: n=10/condition vs. the paper's 500-7405/condition. Treat this section as a pipeline-correctness check (methods run, backoffs fire, numbers are in a plausible range) rather than a statistically powered comparison. The n=100 scale-up is still pending.

2. Why token usage was front-loaded

Token/tool-call volume was heavy in Milestones 0-1 and dropped sharply from Milestone 2 onward. The `AI_LOG.md` issue trail shows why — it's structural, not random:

Phase	Work type	Issues hit	Token driver
M0 (LLM client)	First contact with <code>z-ai/glm-5.2</code>	Hidden reasoning tokens silently eating <code>max_tokens; n>1</code> silently ignored by provider	Both were silent-failure classes — no exception, no error, just wrong/empty output. Each required raw-API inspection, hypothesis, a fix, and a live re-verification round-trip.
M1 (HotpotQA, from scratch)	New env (<code>wiki_env.py</code>), new prompts, new dataset acquisition	Windows path mangling in <code>makedirs</code> ; Wikipedia 403 (no User-Agent); nav-chrome leaking into scraped text; dataset host unreachable (proxy interception), needed HF Hub API discovery; undocumented Act-prompt assumption	Every piece of infrastructure was being built and validated against live external systems (Wikipedia, HF Hub, a paper's official repo) for the first time — each had its own failure mode that could only be found by running it.

M2 (FEVER)	Reuse of M1's WikiEnv/agent/strategies	Only 2 new issues (instruction/query-label mismatch, <code>standard()</code> needing a label-format prompt)	Scaffolding (env, agent loop, LLM client, hybrid back-off logic) was already proven. New work was parametrizing existing functions, not discovering new failure modes.
Charts	Pure data aggregation over already-validated result files	1 issue (method-key naming mismatch), caught by inspecting output, not by a crash	No live external calls, no new infra — just reading JSONL already on disk and plotting it.

The pattern: **token cost tracks number of never-before-exercised integration points, not lines of code written**. M0-M1 touched an LLM provider's undocumented quirks, a scraping target's bot-defenses, and a dataset host's availability — three independent external systems, each requiring iterative live debugging since the failures were silent (wrong output, not a stack trace) and could only be caught by manually inspecting results. Once WikiEnv/agent/strategies/llm.py were validated once (M1), every subsequent domain (M2/FEVER) or analysis step (charts) only needed to reuse or lightly parametrize known-good code, so verification collapsed to "does the existing test/gate still pass" instead of "what new thing is broken now."

Practical implication for the remaining n=100 scale-up: it should be cheap for the same reason — same code path, same already-debugged infra, just a longer loop. Any renewed token spike there would itself be a signal that something in the reused pipeline doesn't hold at higher n (e.g. rate limits, a rarer parsing edge case) rather than expected steady-state cost.